

WEEK 12 · LAB SESSION · CAPSTONE

Natural Language → SQL

Use an AI assistant to write SQL queries from plain English — and learn to spot when it gets things wrong.

OVERVIEW

What you'll do today

In this final lab, you will use a free AI assistant to translate questions written in English into SQL queries against a small university enrolment database. You will run those queries, verify whether they return the right answer, and learn the failure modes you should always check for when AI writes code for you.

By the end of this session you will know how to use an AI assistant effectively as a junior developer would — drafting fast, verifying always.

BY THE END OF THIS LAB YOU WILL BE ABLE TO

1. Write effective natural-language prompts that produce correct SQL.
2. Identify the four most common failure modes of AI-generated SQL.
3. Decide when to trust an AI's output, and when to rewrite it yourself.

BEFORE YOU START

Setup

Pick **one** AI assistant. Any of these work — they are all free for casual use. Open it in a browser tab; you do not need to install anything.

→ **Claude (recommended for this lab)**

<https://claude.ai>

→ **ChatGPT**

<https://chat.openai.com>

→ **Google Gemini**

<https://gemini.google.com>

You will also need a SQL environment to **run** the queries the AI gives you. The simplest option is the online sandbox below — no install, no account required.

→ **SQLite Online — paste schema, run queries**

<https://sqliteonline.com>

WHY WE ARE NOT USING MICROSOFT ACCESS TODAY

Access syntax differs in small annoying ways from standard SQL. AI assistants are trained mostly on standard SQL (SQLite, MySQL, PostgreSQL). For this lab we want the AI to look as good as it can — the point is to study where it *still* fails.

YOUR DATASET

The Institute of Computing schema

Below is a small university enrolment database. It tracks students, the courses they take, the lecturers who teach them, and the grades earned. Copy this entire block and paste it into the SQL sandbox to set everything up.

```
-- Institute of Computing – minimal schema for Week 12 lab
CREATE TABLE Departments (
    dept_id      INTEGER PRIMARY KEY,
    dept_name    TEXT NOT NULL,
    building     TEXT NOT NULL
);

CREATE TABLE Lecturers (
    lecturer_id  INTEGER PRIMARY KEY,
    full_name    TEXT NOT NULL,
    email        TEXT UNIQUE,
    dept_id      INTEGER REFERENCES Departments(dept_id),
    rank         TEXT CHECK (rank IN ('Assistant','Associate','Professor'))
);

CREATE TABLE Students (
    student_id   INTEGER PRIMARY KEY,
    full_name    TEXT NOT NULL,
    email        TEXT UNIQUE,
    enrolled_on  DATE NOT NULL,
    dept_id      INTEGER REFERENCES Departments(dept_id),
    year_of_study INTEGER CHECK (year_of_study BETWEEN 1 AND 4)
);

CREATE TABLE Courses (
    course_id    INTEGER PRIMARY KEY,
    course_code  TEXT NOT NULL,
    title        TEXT NOT NULL,
    lecturer_id  INTEGER REFERENCES Lecturers(lecturer_id),
    credits      INTEGER NOT NULL,
    semester     TEXT CHECK (semester IN ('Autumn','Spring'))
);

CREATE TABLE Enrolments (
    enrolment_id INTEGER PRIMARY KEY,
    student_id   INTEGER REFERENCES Students(student_id),
    course_id    INTEGER REFERENCES Courses(course_id),
    enrolled_at  DATE NOT NULL
);

CREATE TABLE Grades (
    grade_id     INTEGER PRIMARY KEY,
    enrolment_id INTEGER REFERENCES Enrolments(enrolment_id),
    mark         REAL NOT NULL,
    submitted_on DATE NOT NULL
);
```

And some sample data so your queries have something to return:

```
INSERT INTO Departments VALUES
(1, 'Computing', 'Building A'),
(2, 'Business', 'Building B'),
(3, 'Psychology', 'Building C');

INSERT INTO Lecturers VALUES
(10, 'Dr Anna Lambrou', 'a.lambrou@academy.edu', 1, 'Professor'),
(11, 'Dr Petros Vlahos', 'p.vlahos@academy.edu', 1, 'Associate'),
(12, 'Dr Maria Tsakiri', 'm.tsakiri@academy.edu', 2, 'Professor'),
(13, 'Dr Yannis Kontos', 'y.kontos@academy.edu', 2, 'Assistant'),
(14, 'Dr Sofia Markou', 's.markou@academy.edu', 3, 'Associate');

INSERT INTO Students VALUES
(101, 'Maria Papadopoulou', 'maria.p@student.edu', '2024-09-15', 1, 2),
(102, 'Nikos Georgiou', 'nikos.g@student.edu', '2025-09-10', 1, 1),
(103, 'Eleni Dimitriou', 'eleni.d@student.edu', '2023-09-20', 2, 3),
(104, 'Kostas Anagnostou', 'kostas.a@student.edu', '2024-09-15', 2, 2),
(105, 'Sofia Tziovas', 'sofia.t@student.edu', '2022-09-12', 3, 4),
(106, 'Dimitris Pavlou', 'dimitris.p@student.edu', '2025-09-10', 1, 1);

INSERT INTO Courses VALUES
(501, 'COMP101', 'Intro to Programming', 10, 15, 'Autumn'),
(502, 'COMP202', 'Database Systems', 11, 15, 'Spring'),
(503, 'BUS101', 'Principles of Management', 12, 10, 'Autumn'),
(504, 'BUS210', 'Marketing Analytics', 13, 15, 'Spring'),
(505, 'PSY110', 'Cognitive Psychology', 14, 15, 'Autumn'),
(506, 'COMP305', 'Machine Learning Basics', 10, 20, 'Spring');

INSERT INTO Enrolments VALUES
(1, 101, 501, '2024-09-25'),
(2, 101, 502, '2025-02-10'),
(3, 102, 501, '2025-09-25'),
(4, 103, 503, '2023-09-28'),
(5, 103, 504, '2024-02-08'),
(6, 104, 503, '2024-09-26'),
(7, 104, 504, '2025-02-12'),
(8, 105, 505, '2022-09-30'),
(9, 101, 506, '2025-02-11'),
(10, 106, 501, '2025-09-25');

INSERT INTO Grades VALUES
(1, 1, 82.0, '2025-01-20'),
(2, 2, 74.5, '2025-06-15'),
(3, 4, 68.0, '2024-01-22'),
(4, 5, 79.0, '2024-06-18'),
(5, 6, 55.5, '2025-01-21'),
(6, 7, 71.0, '2025-06-16'),
(7, 8, 88.0, '2023-01-19'),
(8, 9, 91.5, '2025-06-17');
```

PART 1 OF 3

Easy questions — does it just work?

Start by giving the AI assistant the schema, then ask three warm-up questions. The goal here is to see how well NL→SQL works when the question is unambiguous.

1 Prime the AI with the schema

In your AI assistant, paste this prompt (replace the schema placeholder with the actual CREATE TABLE block from the previous page):

```
I have a SQLite database with this schema. From now on, when I ask a
question in English, give me ONLY the SQL query needed to answer it,
with no explanation. Use standard SQL.
```

```
[paste the CREATE TABLE statements here]
```

2 Ask three simple questions

Type each of these questions to the AI. Run the SQL it gives you in the SQLite sandbox, then complete the answer rows below each question. These are warm-up questions — the AI should get them right. Your job is to verify that, not to assume it.

Q1 How many students do we have in total?

a) AI's query

b) Result

c) Correct? Yes / No

d) If wrong — why? (1 sentence)

Q2 List all courses offered in the Spring semester.

a) AI's query

b) Result

c) Correct? Yes / No

d) If wrong — why? (1 sentence)

Q3 What is the average mark across all submitted grades?

a) AI's query

b) Result

c) Correct? Yes / No

d) If wrong — why? (1 sentence)

TIP — BE SPECIFIC WHEN THE QUESTION IS AMBIGUOUS

Q3 says "the average mark" — but should we average per-grade, or first compute a per-student average and then average those? Different interpretations give different numbers. Run the AI's first answer, then ask it the question a second way and compare. Notice when the AI picks for you instead of asking what you meant.

PART 2 OF 3

Trickier questions — where it starts to slip

Now ask the harder questions. These all require **joins**, **aggregation**, or careful interpretation of what the user means. This is where AI assistants often produce SQL that runs without errors but returns the wrong answer.

3 Write your version first, then ask the AI

For each of the questions below, do this in order:

1. Cover the AI's chat window. Try to write the SQL yourself first — even if you are not sure it is right. Two minutes per question is enough.
2. Now ask the AI. Compare its query with yours. Run both. Note the differences.
3. Fill in the answer card. If the AI's query is wrong, write a fixed version in row (e). The fix can be the AI's query with corrections, or your own version, or something in between.

WHY WE ARE DOING IT THIS WAY

Comparing your draft against the AI's draft is how senior engineers actually use these tools. They never accept the first answer — they argue with it. You can't argue with a query you didn't try to write.

4 The four core questions

Q4 Which lecturer teaches the most courses this academic year?

a) AI's query

b) Result

c) Correct? Yes / No

d) If wrong — why? (1 sentence)

e) Fixed query

Q5

Show me the average mark per course, sorted highest to lowest.

a) AI's query

b) Result

c) Correct? Yes / No

d) If wrong — why? (1 sentence)

e) Fixed query

Q6

Which courses are most popular (have the most enrolments)?

a) AI's query

b) Result

c) Correct? Yes / No

d) If wrong — why? (1 sentence)

e) Fixed query

Q7

List students who enrolled in 2025 but have no grades recorded yet.

a) AI's query

b) Result

c) Correct? Yes / No

d) If wrong — why? (1 sentence)

e) Fixed query

WATCH FOR THESE FOUR FAILURE MODES

1. **Wrong join logic** — uses LEFT JOIN where INNER was needed (or vice versa).
2. **Counting duplicates** — forgets DISTINCT and inflates the count.
3. **Made-up columns** — references a column that doesn't exist in your schema.
4. **Plausible but wrong logic** — the SQL runs and returns numbers, but the numbers do not actually answer your question.

5 Adversarial round — give it a vague question

Ask the AI exactly this, with no further detail:

`Show me our best students.`

What did it pick? Best by what — average mark, number of courses, both? Did it ask you to clarify, or did it just guess? Write the assumption the AI made, and one alternative interpretation it should have asked about.

6 Bonus — only if you finish early

This question is harder than anything above. It needs you to think in two steps: first, find something true for **every** course a student is enrolled in; second, list the students for whom that holds. Don't ask the AI yet — try to write it yourself. Then compare.

Q8

Bonus. List students who have *passed* every course they have a grade for. (A pass is mark ≥ 50 .) Include their full name and department.

a) AI's query

b) Result

c) Correct? Yes / No

d) If wrong — why? (1 sentence)

e) Fixed query

PART 3 OF 3

The dangerous one

This last task is short but important. It demonstrates the most common way AI-generated SQL causes real damage in production.

7 Ask for a destructive operation

Ask the AI assistant:

```
Delete all student records who haven't enrolled in any course in the last year.
```

Read the SQL it produced — but **do not run it yet**. Answer these in writing on your answer sheet:

- Did the AI add any safety check (e.g. preview the rows it would delete first)?
- What would happen to a final-year student on a leave of absence who hasn't enrolled this year?
- What about the existing grades and enrolment history of those students — would they be left dangling?
- If you ran this query, could you undo it?

NEVER RUN DESTRUCTIVE AI-GENERATED SQL ON REAL DATA

Always test on a copy. The SQL the AI gave you is a *draft*, not a decision. The decision to run it is always yours.

8 Three ways professionals stay safe

When you have to run a destructive query in real life, you never just fire it. You wrap it in one of these three patterns. Read each, then answer the question at the end.

Pattern 1 — Preview before delete

Run the same WHERE clause as a SELECT first. Read the rows. If they look right, only then run the DELETE.

```
-- Step 1: PREVIEW
SELECT * FROM Students
WHERE student_id NOT IN (
    SELECT student_id FROM Enrolments
    WHERE enrolled_at >= date('now', '-1 year')
);

-- Step 2: only after eyeballing the results
DELETE FROM Students
WHERE student_id NOT IN (...same WHERE clause...);
```

Pattern 2 — Soft delete

Don't actually delete the row. Add an archived_on column and set a date. The data is hidden from normal queries but recoverable.

```
ALTER TABLE Students ADD COLUMN archived_on DATE;

UPDATE Students
SET archived_on = date('now')
WHERE student_id NOT IN (...same WHERE clause...);
```

Pattern 3 — Backup first

Copy the rows into a backup table before deleting them. If something goes wrong, you can restore.

```
CREATE TABLE Students_archive AS
  SELECT * FROM Students
  WHERE student_id NOT IN (...same WHERE clause...);

DELETE FROM Students
WHERE student_id NOT IN (...same WHERE clause...);
```

YOUR TURN

Which of the three patterns would you use for the student-deletion task, and why? Write 2–3 sentences. Consider: how easily could a mistake be reversed? What would happen if a student had grades from previous years that you still need for transcripts?

FINAL CHALLENGE

Design a question that breaks the AI

You have spent the lab seeing where AI assistants fail. Now turn it around. Write a question in plain English that you believe would trick the AI into producing wrong SQL against this schema. Then test it. Did it work? If not, refine your question.

C1 **Your trick question:** write the question you'll ask, the SQL the AI returned, why it is wrong, and what fix you would apply.

a) AI's query

b) Result

c) Correct? Yes / No

d) If wrong — why? (1 sentence)

e) Fixed query

CLOSING

Reflection — to discuss in the last 10 minutes

We will close the session with a short group discussion. Come prepared to answer:

1. Of the four core questions in Part 2, how many did the AI answer correctly on the first try?
2. Which type of failure surprised you most?
3. When you wrote your own SQL first, did you spot a mistake in the AI's version that you might have missed otherwise?
4. Knowing what you saw today — would you trust an AI to write production SQL? Under what conditions?
5. How does this change the value of the SQL skills you built this year?

SUBMISSION

Nothing to submit. This lab is formative — your CW2 remains the assessed deliverable for this module.

Thank you for a great year.